



AI Driven SQA: Manual & Automation Testing

Assignment 05

Module: 05

Topic: Problem Solving with AI (Claude Code + JavaScript)

Submitted by,

Name: Jhion Adbar Opi

Batch: 21

Phone: 01778812373

Q1. JavaScript Coding Practice.

01. Convert Celsius to Fahrenheit.

Answer: The program accepts a Celsius temperature from the user through the Node.js command-line interface. Convert the input into a numerical value. Applies the Celsius-to-Fahrenheit conversion formula and displays the resulting Fahrenheit temperature.

Output:

```
JS c_to_f.js X
JS c_to_f.js > ...
1 // Celsius to Fahrenheit
2 // Jhion Adbar Opi Batch 21
3
4 const readline = require("readline");
5 // "readline" holo built in Node.js module
6
7 const rl = readline.createInterface({
8   //createInterface() interface use kori input and output read korar jonno
9   input: process.stdin,
10  output: process.stdout
11 });
12
13 rl.question("Enter temperature in Celsius: ", (celsius) => {
14   let fahrenheit = (parseFloat(celsius) * 9 / 5) + 32;
15   // parseFloat() use hoise decimal values handel ar jonno.
16
17   console.log("Temperature in Fahrenheit:", fahrenheit);
18
19   rl.close();
20 });
```

Problems Output Debug Console Terminal Ports

```
● PS E:\SQA\Module_5_by Jhion Adbar Opi> node c_to_f.js
Enter temperature in Celsius: -56525.365
Temperature in Fahrenheit: -101713.65699999999
○ PS E:\SQA\Module_5_by Jhion Adbar Opi> █
```

Source Code:

https://drive.google.com/drive/folders/1gZiRwN0ktX8Zn8BDXpPTg4JLdlr_5CZI?usp=sharing

Explanation:

[readline] Module: The program imports Node.js's built-in [readline] module using require(). This module is used to receive input from the user through the command-line interface.

Creating the [readline] Interface: [readline.createInterface()] creates an interface for handling input and output. [process.stdin] is used to receive input from the keyboard. While [process.stdout] is used to display output in the terminal.

Taking User Input: The [rl.question()] method prompts the user to enter a temperature in Celsius. The entered value is received by the callback function through the [celsius] parameter.

Converting Input to a Number: Since command-line input is received as a string. [parseFloat()] converts the input into a floating-point number. This allows the program to handle positive, negative and decimal temperature values.

Temperature Conversion: The program converts Celsius to Fahrenheit using the formula:

$$F = (C \times \frac{9}{5}) + 32$$

The calculated value is stored in the [fahrenheit] variable.

Displaying the Result: [console.log()] displays the converted Fahrenheit temperature in the terminal.

Closing the Interface: [rl.close()] closes the [readline] interface and terminates the input operation. Allowing the program to exit properly.

02. Find the Factorial of a Number.

Answer: The program uses a user-defined function and an iterative for loop to calculate the factorial of a non-negative integer. User input is collected using Node.js's [readline] module and converted into an integer using [parseInt()]. The program validates the input and calculates the factorial through repeated multiplication. Then displays the result and finally closes the [readline] interface.

Output:

```
// Find the Factorial of a Number
// Jhion Adbar Opi Batch 21

const readline = require("readline");

const rl = readline.createInterface({
  input: process.stdin, // receive input
  output: process.stdout // display output in terminal
});

function factorial(n) {
  let result = 1;

  for (let i = 1; i <= n; i++) {
    result = result * i;
  }

  return result;
}

rl.question("Enter a non-negative integer: ", (input) => {

  let number = parseInt(input);

  if (number < 0 || isNaN(number)) {
    console.log("Please enter a valid non-negative integer.");
  } else {
    let result = factorial(number);
    console.log("Factorial of", number, "is:", result);
  }

  rl.close();
});
```

Problems Output Debug Console **Terminal** Ports

- PS E:\SQA\Module_5_by Jhion Adbar Opi> node factorial.js
Enter a non-negative integer: 25862
Factorial of 25862 is: Infinity
- PS E:\SQA\Module_5_by Jhion Adbar Opi> node factorial.js
Enter a non-negative integer: 10
Factorial of 10 is: 3628800
- PS E:\SQA\Module_5_by Jhion Adbar Opi>

Source Code:

https://drive.google.com/drive/folders/1uAnVFZ81YPQ4Qr0Q5gncipj9zt96wA7T?usp=drive_link

Explanation:

Importing the [readline] Module: The program imports Node.js's built-in readline module to receive input from the user through the command-line interface.

Creating the [readline] Interface: [readline.createInterface()] creates an interface for handling input and output. [process.stdin] receives input from the keyboard. While [process.stdout] displays output in the terminal.

Defining the Factorial Function: The factorial(n) function is created to calculate the factorial of the given number. The parameter n represents the input number.

Initializing the Result: The variable result is initialized with 1. This is necessary because factorial multiplication starts with 1.

Using a For Loop: A for loop iterates from 1 to the given number. During each iteration, the current value of i is multiplied with result.

result = result * i;

Returning the Factorial: After the loop completes, the calculated factorial is returned using the return statement.

Taking User Input: The [rl.question()] method asks the user to enter a non-negative integer. The input is initially received as a string.

Converting Input to an Integer: [parseInt()] converts the user's input from a string into an integer so that mathematical operations can be performed.

Input Validation: The program checks whether the entered value is negative or invalid using:

if (number < 0 || isNaN(number))

Factorial is defined for non-negative integers, so negative or invalid input is rejected.

Calling the Function: If the input is valid. The factorial() function is called with the user's number and the returned result is stored in the result variable.

Displaying the Result: [console.log()] displays the calculated factorial in the terminal.

Closing the Interface: [rl.close()] closes the [readline] interface after the program finishes processing the input.

03. Check for Palindrome.

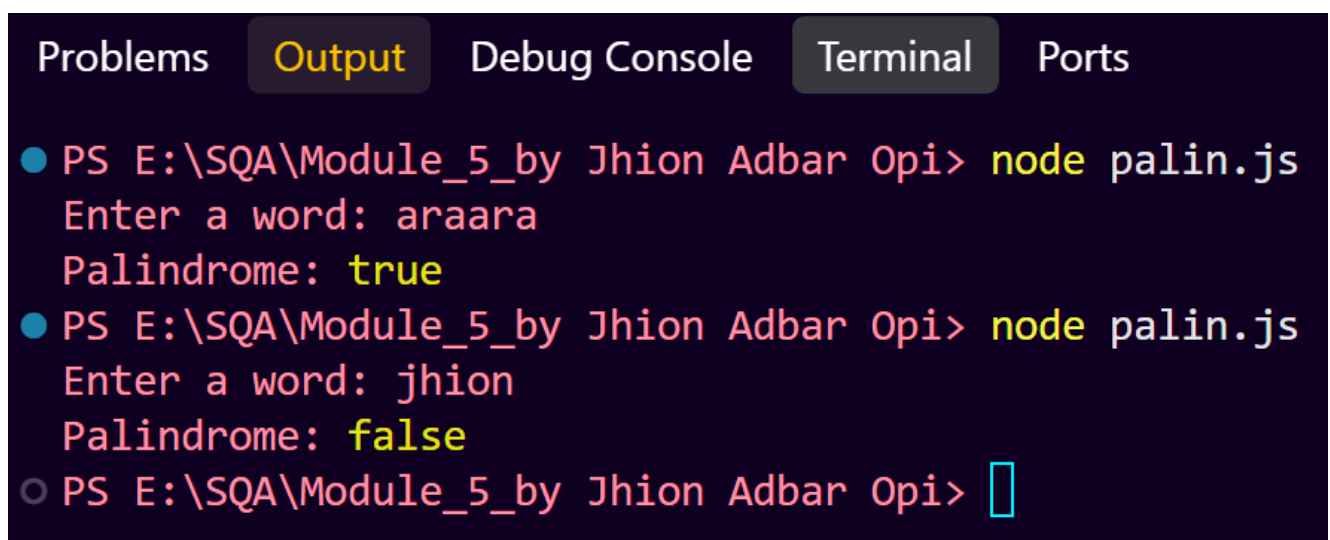
Answer: The program uses a user-defined function to determine whether a given word is a palindrome. The input string is converted into an array of characters using `split()`, reversed using `reverse()` and converted back into a string using `join()`. The reversed string is then compared with the original string using the strict equality operator `[ === ]`. The function returns a Boolean value `[ true ]` for a palindrome and `[ false ]` otherwise.

Output:



```
JS palin.js X JS c_to_f.js X JS factorial.js X

JS palin.js > ...
1 // Check for Palindrome
2 // Jhion Adbar Opi Batch 21
3
4 const readline = require("readline");
5
6 const rl = readline.createInterface({
7   input: process.stdin,
8   output: process.stdout
9 });
10
11 function isPalindrome(word) {
12   let reversedWord = word.split("").reverse().join("");
13
14   return word === reversedWord;
15 }
16
17 rl.question("Enter a word: ", (word) => {
18
19   let result = isPalindrome(word);
20
21   console.log("Palindrome:", result);
22
23   rl.close();
24 });
```



```
Problems Output Debug Console Terminal Ports

● PS E:\SQA\Module_5_by Jhion Adbar Opi> node palin.js
Enter a word: araara
Palindrome: true
● PS E:\SQA\Module_5_by Jhion Adbar Opi> node palin.js
Enter a word: jhion
Palindrome: false
○ PS E:\SQA\Module_5_by Jhion Adbar Opi> 
```

Source Code: https://drive.google.com/drive/folders/1GtAuIIM3O1ZAMKazwZncpA-UDNuvGGtE?usp=drive_link

Explanation:

Importing the [readline] Module: The program imports Node.js's built-in [readline] module to receive input from the user through the command-line interface.

Creating the [readline] Interface: [readline.createInterface()] creates an interface for handling input and output. [process.stdin] receives keyboard input, while [process.stdout] displays output in the terminal.

Defining the Palindrome Function: The [isPalindrome(word)] function is created to determine whether the given word is a palindrome. The [word] is passed to the function through the word parameter.

Splitting the String: The [split("")] method separates the string into individual characters.

Output we get : "araara" => ["a", "r", "a", "a", "r", "a"]

Reversing the Characters: The [reverse()] method reverses the order of the characters.

["a", "r", "a", "a", "r", "a"] => ["a", "r", "a", "a", "r", "a"]

In the case of [araara], the reversed word is identical to the original word.

Joining the Characters: The [join("")] method combines the individual characters back into a single string. [word.split("").reverse().join("")]; This produces the reversed version of the original word.

Comparing the Original and Reversed Words: The program uses the strict equality operator [===]:
[return word === reversedWord;]

If both strings are identical, the function returns true. Otherwise, it returns false.

Taking User Input: The [rl.question()] method asks the user to enter a word. The entered word is passed to the callback function through the word parameter.

Calling the Function: The [isPalindrome()] function is called with the user's input, and the returned Boolean value is stored in the result variable.

Displaying the Result: [console.log()] displays whether the entered word is a palindrome.

Closing the [readline] Interface: [rl.close()] closes the [readline] interface after the program finishes processing the input.

04. Sum of Array Elements.

Answer: The program accepts multiple numerical values from the user, converts the input into an array of numbers and passes the array to a user-defined function. The function uses a for loop to iterate through each array element and accumulates their values in a sum variable. Then the function returns the total sum. Which is displayed in the terminal.

Output:

```
// Sum of Array Elements
// Jhion Adbar Opi Batch 21

const readline = require("readline");

const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout
});

function sumArray(numbers) {
  let sum = 0;

  for (let i = 0; i < numbers.length; i++) {
    sum = sum + numbers[i];
  }

  return sum;
}

rl.question("Enter numbers separated by spaces: ", (input) => {

  let numbers = input.split(" ").map(Number);

  let result = sumArray(numbers);

  console.log("Sum of array elements:", result);

  rl.close();
});
```

Problems Output Debug Console **Terminal** Ports

```
● PS E:\SQA\Module_5_by Jhion Adbar Opi> node sum_arr.js
  Enter numbers separated by spaces: 25 699 487 52 54 524
  Sum of array elements: 1841
○ PS E:\SQA\Module_5_by Jhion Adbar Opi> █
```

Source Code:

https://drive.google.com/drive/folders/1mVnF9m8JtIYYKphjtMMbCJ_t3K-LM6of?usp=drive_link

Explanation:

Importing the [readline] Module: The program imports Node.js's built-in [readline] module to receive input from the user through the command-line interface.

Creating the [readline] Interface: [readline.createInterface()] creates an interface for handling input and output. [process.stdin] receives input from the keyboard. While [process.stdout] displays output in the terminal.

Defining the Function: The [sumArray(numbers)] function is created to calculate the sum of all elements in an array. The array is passed to the function through the [numbers] parameter.

Initializing the Sum: The variable sum is initialized to 0. [let sum = 0;]. Starting with zero allows each array element to be added to the accumulated total.

Iterating Through the Array: A for loop is used to access each element of the array.

for (let i = 0; i < numbers.length; i++)

The loop starts from index 0 and continues until it reaches the last element of the array.

Adding Array Elements: Each element is added to the sum variable:

sum = sum + numbers[i];

The value of sum is updated during every iteration.

Returning the Result: After all elements have been processed the [return] statement returns the calculated sum.

Taking User Input: The [rl.question()] method asks the user to enter numbers separated by spaces.
Given Output:

Enter numbers separated by spaces: 10 20 30 40

Converting Input into an Array: The [split(" ")] method separates the input string at each space.

input.split(" ")

This produces:

["10", "20", "30", "40"]

Converting Strings to Numbers: The [map(Number)] method converts each string element into a numerical value.

.map(Number)

Therefore:

["10", "20", "30", "40"]

becomes:

[10, 20, 30, 40]

Calling the Function: The [sumArray()] function is called with the converted array, and the returned value is stored in the result variable.

Displaying the Result: [console.log()] displays the calculated sum in the terminal.

Closing the Interface: [rl.close()] closes the [readline] interface after the program finishes processing the input.

05. FizzBuzz (Basic).

Answer: The program implements a flexible FizzBuzz algorithm using Node.js and user-defined input. The user provides a starting number and the desired sequence length. A for loop then processes exactly the specified number of consecutive integers. The modulus operator (%) determines divisibility by 3 and 5. While conditional statements produce [Fizz, Buzz, FizzBuzz] or the original number accordingly. This approach makes the basic FizzBuzz problem reusable for different starting values and sequence lengths.

Code:

```
JS fizzbuzz.js X JS sum_arr.js X JS palin.

JS fizzbuzz.js > rl.question("Enter the startin

1 // FizzBuzz
2 // Jhion Adbar Opi Batch 21
3
4 const readline = require("readline");
5
6 const rl = readline.createInterface({
7   input: process.stdin,
8   output: process.stdout
9 });
10
11 rl.question("Enter the starting number: ", (startInput) => {
12
13   rl.question("Enter the length: ", (lengthInput) => {
14
15     let start = parseInt(startInput);
16     let length = parseInt(lengthInput);
17
18     for (let i = start; i < start + length; i++) {
19
20       if (i % 3 === 0 && i % 5 === 0) {
21         console.log("FizzBuzz");
22       }
23       else if (i % 3 === 0) {
24         console.log("Fizz");
25       }
26       else if (i % 5 === 0) {
27         console.log("Buzz");
28       }
29       else {
30         console.log(i);
31       }
32     }
33
34     rl.close();
35   });
36 });
```

Output:

```
Problems  Output  Debug Console  Terminal  Ports

● PS E:\SQA\Module_5_by Jhion Adbar Opi> node fizzbuzz.js
  Enter the starting number: 1
  Enter the length: 15
  1
  2
  Fizz
  4
  Buzz
  Fizz
  7
  8
  Fizz
  Buzz
  11
  Fizz
  13
  14
  FizzBuzz
○ PS E:\SQA\Module_5_by Jhion Adbar Opi> █
```

Source Code:

https://drive.google.com/drive/folders/1B9Ce1Yr3iwSFmcBZMj-sq9Uh8qB02owk?usp=drive_link

Explanation:

Taking the Starting Number: The program asks the user to provide the starting number. This value determines where the FizzBuzz sequence begins.

Taking the Length: The program also asks the user to enter the length of the sequence. The length determines how many numbers will be processed.

Converting Input to Integers: [parseInt()] converts the input received from the [readline] interface from strings into integer values.

Controlling the Loop: The for loop starts from the user-provided start value and executes for the specified length.
for (let i = start; i < start + length; i++)

The condition [i < start + length] ensures that exactly the requested number of values is processed.

Checking Divisibility by Both Numbers: The first condition checks whether the current number is divisible by both 3 and 5.

i % 3 === 0 && i % 5 === 0

If both conditions are true, the program prints [FizzBuzz].

Checking Divisibility by 3: If the number is not divisible by both. The program checks whether it is divisible by 3. If true, it prints [Fizz].

Checking Divisibility by 5: If the number is not divisible by 3. The program checks whether it is divisible by 5. If true, it prints [Buzz].

Printing the Original Number: If none of the divisibility conditions are satisfied. The original number is printed.

Closing the Interface: [rl.close()] closes the [readline] interface after the loop has completed.